

Snoopy HQ Journal — Project Handbook

Landed here first? This is the deep architecture reference — the third thing most people should read. If the project is new to you, start with [GETTING-STARTED.md](#) (setup, vocabulary, first article, ~45 min). If you only need to publish or edit content, you want [MAINTAINER-MANUAL.md](#) instead. All documents are routed from the [documentation index](#).

Audience: anyone who has never seen this repository before — a new maintainer, a reviewer, or a future you. This document explains *what the project is, how every moving part works, why it was built that way, and how to keep it alive*.

Companion document: [MAINTAINER-MANUAL.md](#) — the short, task-oriented "how do I add a blog post" manual. Read this handbook once; use the manual daily.

Table of contents

1. What this project is
2. Technology stack and why each piece is here
3. How a request is served (runtime architecture)
4. Repository map
5. The content model
6. The content registry pattern
7. Where content actually comes from (three sources)
8. The article import pipeline
9. The image pipeline
10. The document (attachment) pipeline
11. Routing and pages
12. Component catalogue — every file, what it does, why
13. Classes vs functions — what is actually a class and why
14. Styling, theming and design tokens
15. Motion and scroll behaviour
16. Accessibility decisions
17. Security decisions
18. Tests
19. Build, run and deploy
20. Managing this project in the future
21. Known gaps, debt and gotchas
22. Glossary

1. What this project is

Snoopy HQ Journal is a **statically-shaped editorial blog** — a magazine-style site with a homepage, category browsing, search, article pages, a full-screen photo viewer, and in-browser previews of attached Office/PDF documents.

Three things make it unusual compared with a typical Next.js blog:

Property	What it means in practice
Content is data, not JSX	An article is a frozen JavaScript object matching the <code>BlogPost</code> type. No component ever hard-codes an article's text, image position, or attachment. Adding content never means editing a component.
Content arrives through a transactional importer	You drop a folder into <code>imports/articles/</code> . A Node script validates it, checksums it, stages it, atomically commits it, and only then deletes your folder. A half-written article can never reach the site.
Everything is deeply frozen and ID-addressed	Every article, image and document has a permanent ID. Registries are built once at module load, validated for duplicates, and <code>Object.freeze</code> d recursively. Runtime code physically cannot mutate published content.

The design goal, stated plainly: **an editor should be able to publish without touching TypeScript, and a bug in one article should never be able to corrupt another.**

2. Technology stack and why each piece is here

From `package.json`:

Runtime dependencies

Package	Version	Why it is here
<code>next</code>	16.2.6	Provides the App Router conventions, <code>Link</code> , <code>next/font</code> , <code>Metadata</code> API, <code>notFound()</code> . The site is written as a Next.js app but is not run by the Next.js server (see <code>vinext</code> below).
<code>react / react-dom</code>	19.2.6	React 19 Server Components. Most components render on the server with zero client JS; only files marked <code>"use client"</code> ship to the browser.
<code>framer-motion</code>	<code>^12</code>	All motion: page fades, card press feedback, scroll reveals, dialog enter/exit. Chosen over hand-written CSS because every animation needed a <code>useReducedMotion()</code> escape hatch, which Framer gives for free.
<code>lucide-react</code>	<code>^0.468</code>	Icon set. Tree-shakeable, stroke-based, inherits <code>currentColor</code> , so icons theme automatically.
<code>@file-viewer/core</code> , <code>@file-viewer/react</code> , <code>@file-viewer/preset-office</code>	<code>^2.2.3</code>	The "Flyfish" viewer. Renders <code>.pptx</code> , <code>.docx</code> , <code>.xlsx</code> , <code>.csv</code> and <code>.pdf</code> in the browser , client-side, with no server round-trip and no third-party upload.
<code>jszip</code>	<code>^3.10</code>	Used by the importer (Node side) to structurally validate OpenXML files, and by the docx renderer in the browser.
<code>buffer</code> , <code>stream-browserify</code> , <code>util</code>	—	Browser shims that JSZip's browser build expects. Aliased in <code>vite.config.ts</code> ; without them Vite logs externalisation warnings and the docx worker fails.
<code>drizzle-orm</code>	0.45.2	Present from the starter template. Currently unused — <code>db/schema.ts</code> is deliberately empty. Kept so a future feature (comments, newsletter storage) can add a Cloudflare D1 table without re-plumbing.

Dev dependencies that matter

Package	Why
<code>vinext</code>	The actual runtime. Compiles the Next.js App Router app into a Cloudflare Worker . This is why there is a <code>worker/index.ts</code> and a <code>dist/server/index.js</code> , and why <code>npm run dev</code> runs <code>vinext dev</code> , not <code>next dev</code> .
<code>@cloudflare/vite-plugin</code> , <code>wrangler</code>	Local Cloudflare emulation (Miniflare): D1/R2 bindings, the assets fetcher, Worker semantics.
<code>@vitejs/plugin-rsc</code> , <code>react-server-dom-webpack</code>	Server Component serialisation across the Vite build.
<code>sharp</code>	Node-side image processing. Generates the AVIF/WebP responsive variants and reads real image dimensions during import.
<code>@file-viewer/vite-plugin</code>	Copies the viewer's Web Workers, WASM, fonts and renderer chunks into <code>public/vendor/</code> so previews work offline and same-origin.
<code>tailwindcss 4+</code> <code>@tailwindcss/postcss</code>	Imported at the top of <code>globals.css</code> . Used mainly for its reset and a handful of utilities; the site's real styling is hand-written CSS with custom properties , not utility soup.
<code>typescript 5.9</code>	Strict typing of the content contract. The <code>BlogPost</code> type is the spine of the whole codebase.
<code>eslint + eslint-config-next</code>	Lint.
<code>drizzle-kit</code>	Migration generation, unused today.

Why not a CMS? Because the content set is small, the authorship is single-person, and the import pipeline gives the two properties a CMS is usually bought for — validation and atomic publishing — without a database, an admin UI, or a hosted dependency. If the site ever needs multi-author concurrent editing, that is the point at which this decision should be revisited.

3. How a request is served (runtime architecture)



Key consequences of this shape:

- **There is no database on the read path.** All article data is compiled into the bundle from `content/index.json` and `app/content/static-articles.ts`. Page rendering is pure CPU.
- **Static assets** (images, documents, viewer workers) are served from `public/` by the Cloudflare assets binding, with headers from `public/_headers`.
- `worker/index.ts` **is thin on purpose**. It intercepts exactly one path, `/_vinext/image`, to use Cloudflare's Images binding for on-the-fly resizing; everything else is delegated. Note that the site's *article* images do **not** use this path — they use the pre-generated `/_optimized/` variants instead (see §9).

```
// worker/index.ts – the entire routing decision
if (url.pathname === "/_vinext/image") {
  return handleImageOptimization(request, { /* ... */ }, allowedWidths);
}
return handler.fetch(request, env, ctx);
```

4. Repository map

```
Blog/
├─ app/                                ← the site (Next.js App Router)
│  ├─ layout.tsx                      root HTML shell, fonts, theme boot script
│  ├─ page.tsx                        homepage
│  ├─ globals.css                     ~600 lines: all design tokens + all component CSS
│  ├─ content.ts                      PUBLIC BARREL – the only import path components use
│  ├─ content/                        the content layer (data, registries, contracts)
│  │  ├─ types.ts                     BlogPost, BlogSection, MediaAsset, DocumentAsset...
│  │  ├─ content-registry.ts          deepFreeze + createContentRegistry factory
│  │  ├─ articles.ts                 merges + sorts + validates all articles
│  │  ├─ static-articles.ts           hand-written built-in articles
│  │  ├─ media-assets.ts              hand-written images + imported image registry
│  │  ├─ document-assets.ts           hand-written documents + imported document registry
│  │  ├─ document-contract.ts         MIME/extension/size invariants for documents
│  │  ├─ document-placements.ts       "which document appears in which section"
│  │  ├─ categories.ts               category ordering + discovery
│  │  └─ local-preview-article.ts     dev-only QA fixture article
│  ├─ blog/[slug]/page.tsx            article route
│  ├─ photo/[id]/page.tsx             full-page photo route
│  └─ components/                     see §12 for the full catalogue
│     ├─ document/                    attachment card + preview dialog + viewer
│     ├─ media/                       responsive <picture>, zoom dialog, zoom controls
│     ├─ motion/                      reveal, card press, page transition, scroll memory
│     ├─ navigation/                  scroll-aware header shell
│     └─ photo/                       photo focus page pieces
│
├─ content/                           ← CANONICAL IMPORTED CONTENT (committed)
│  ├─ index.json                      the registry: ids, slugs, articles, assets, documents
│  ├─ articles/<articleId>.json        one resolved article per file
│  └─ assets/<articleId>/<assetId>.json image metadata + sha256
│
├─ imports/                           ← THE INBOX (you put things here)
│  ├─ articles/<package>/              article.json or article.md + images/ + documents/
│  └─ <batch>/import-ready/<pkg>/     portable batch layout
│
├─ public/                            ← served verbatim
│  ├─ images/...                      original images (hand-placed and imported)
│  ├─ documents/...                   original attachments
│  ├─ _optimized/                     GENERATED avif/webp variants (gitignored)
│  ├─ vendor/                         GENERATED viewer workers/wasm (gitignored)
│  └─ _headers                        Cloudflare header rules
```

```
|
├─ scripts/                                ← the Node-side pipeline
|   ├─ import-articles.mjs                CLI entry
|   ├─ watch-article-imports.mjs          filesystem watcher for `npm run dev`
|   ├─ dev-with-imports.mjs               supervises watcher + dev server together
|   ├─ optimize-images.mjs                sharp responsive variant generator
|   └─ article-import/
|       ├─ run-imports.mjs                 discovery + locking + orchestration
|       ├─ import-package.mjs              validation → staging → atomic commit
|       ├─ validation.mjs                  the schema validator (whitelist, strict)
|       ├─ markdown-package.mjs            article.md → article.json adapter
|       └─ filesystem.mjs                  path-safety + atomic write helpers
|
├─ tests/
|   ├─ article-import.test.mjs             importer behaviour (temp dirs, real files)
|   └─ rendered-html.test.mjs              renders the built Worker and asserts on HTML
|
├─ worker/index.ts                         Cloudflare Worker entry
├─ vite.config.ts                          plugins: viewer assets, dev middlewares, cloudflare
├─ build/sites-vite-plugin.ts               hosting-platform plugin
├─ db/                                      empty Drizzle schema (reserved)
├─ design-system/.../MASTER.md              original design brief (historical reference)
└─ .article-import/                        GENERATED lock, journals, receipts (gitignored)
```

5. The content model

Everything hangs off `app/content/types.ts`. Read this file first; the rest of the codebase is downstream of it.

BlogPost

```
export type BlogPost = {
  readonly id: string;           // permanent identity, never changes
  readonly slug: string;         // URL: /blog/<slug>
  readonly featuredRank?: number; // reserved; ordering is currently by date
  readonly category: string;     // free text; drives the category browser
  readonly title: string;
  readonly summary: string;       // used on cards, search, <meta description>
  readonly date: string;          // "21 July 2026" – human format, parsed on load
  readonly author: string;
  readonly tags: readonly string[];
  readonly accent: "sky" | "coral" | "teal" | "navy"; // CSS accent class
  readonly art: ArtVariant;       // fallback illustration when no image exists
  readonly artLabel?: string;     // text drawn inside that illustration
  readonly heroImage?: BlogImage; // top-of-article photo
  readonly kicker?: string;       // lead-in paragraph above section 1
  readonly sections: readonly BlogSection[];
  readonly related: readonly string[]; // article IDs *or* slugs
};
```

Two deliberate choices worth understanding:

- **id and slug are separate.** The slug is the URL and may be rewritten for SEO; the id is permanent and is what images, documents, placements and related-article links point at. Renaming a slug does not break any internal reference.
- **date is a human string, not an ISO date.** It is the exact text shown to readers. Machine ordering is derived from it at load time by `publicationTime()` in `articles.ts`, which throws loudly on a malformed date rather than silently sorting wrongly:

```
function publicationTime(date: string) {
  const match = /^(\d{1,2}) ([A-Za-z]+) (\d{4})$/.exec(date);
  if (!match || monthNumbers[match[2]] === undefined) {
    throw new Error(`Invalid article date "${date}". Use the format "21 July 2026".`);
  }
  return Date.UTC(Number(match[3]), monthNumbers[match[2]], Number(match[1]));
}
```

BlogSection

A section is a chunk of the article with its own anchor, heading, paragraphs, and optional extras:

```
export type BlogSection = {
  readonly id: string; // kebab-case anchor, used by the TOC
  readonly title: string; // rendered as <h2>
  readonly paragraphs: readonly string[];
  readonly quote?: string; // pull-quote at the end of the section
  readonly list?: readonly BlogListItem[];
  readonly listStyle?: "ordered" | "unordered";
  readonly references?: readonly BlogReference[]; // external links list
  readonly images?: readonly BlogImage[];
  readonly documents?: readonly BlogDocument[];
};
```

Placement: the afterParagraph convention

This is the single most important idea for content authors. An image or document is not "inside" a paragraph — it carries a number saying **which paragraph it follows**:

afterParagraph	Renders
-1	Immediately after the section <code><h2></code> , before any paragraph
0	After the first paragraph
2	After the third paragraph

`BlogImage` and `BlogDocument` are simply the asset type plus that number:

```
export type BlogImage = MediaAsset & { readonly afterParagraph: number };
export type BlogDocument = DocumentAsset & { readonly afterParagraph: number };
```

And `ArticleBody` renders it with a filter per position — this is the whole layout engine:

```
// app/components/article-body.tsx
{section.images?.filter((image) => image.afterParagraph === -1)
  .map((image) => <ArticlePhoto image={image} key={image.src} />)}

{section.paragraphs.map((paragraph, paragraphIndex) => (
  <Fragment key={`_${section.id}-${paragraphIndex}`}>
    <p>{renderInlineEmphasis(paragraph)}</p>
    {section.images?.filter((image) => image.afterParagraph === paragraphIndex)
      .map((image) => <ArticlePhoto image={image} key={image.src} />)}
    {section.documents?.filter((d) => d.afterParagraph === paragraphIndex)
      .map((d) => <ArticleDocument document={d} key={d.id} />)}
  </Fragment>
)}}}
```

Inline emphasis

Paragraph text is plain text with exactly one piece of markup: `*asterisks*` become `<em>`. Nothing else — no raw HTML, no links inside paragraphs (use `references`).

```
function renderInlineEmphasis(text: string) {
  return text.split(/(\*[^\*]+\*)/g).map((part, index) => (
    part.startsWith("*") && part.endsWith("*")
      ? <em key={index}>{part.slice(1, -1)}</em>
      : part
  ));
}
```

That restriction is intentional: article text comes from an importer, and permitting HTML would mean permitting injected markup from a file dropped into a folder.

MediaAsset **and** DocumentAsset

```
export type MediaAsset = {
  readonly id: string;           // asset identity – also the /photo/<id> URL
  readonly articleId: string;    // owner article
  readonly title: string;        // shown on the photo page and zoom dialog
  readonly src: string;          // "/images/tokyo/skytree-view.jpg"
  readonly width: number;        // REAL pixel dimensions – prevents layout shift
  readonly height: number;
  readonly alt: string;          // required, never optional
  readonly caption: string;      // shown under the photo
  readonly articleSlug: string;
  readonly portrait?: boolean;   // switches to the tall layout + sizes hint
};

export type DocumentAsset = {
  readonly id: string;
  readonly articleId: string;
  readonly articleSlug: string;
  readonly title: string;
  readonly filename: string;      // original filename, used for download
  readonly src: string;           // "/documents/.../file.xlsx"
  readonly extension: "pptx" | "docx" | "xlsx" | "csv" | "pdf";
  readonly mimeType: string;      // must match the extension exactly
  readonly size: number;          // bytes, shown on the card
  readonly caption: string;
};
```

`width` / `height` / `size` are not decoration — they are load-bearing. Width and height give the `<img>` an intrinsic aspect ratio (no cumulative layout shift), and `size` is passed to the viewer so it can budget its parse.

6. The content registry pattern

`app/content/content-registry.ts` is 50 lines and is used by all three registries (articles, media, documents). It does three jobs.

```
export function deepFreeze<T>(value: T): T {
  if (value === null || typeof value !== "object" || Object.isFrozen(value)) return value;
  for (const nestedValue of Object.values(value)) deepFreeze(nestedValue);
  return Object.freeze(value);
}

export function createContentRegistry<T extends RegistryEntry>(
  source: readonly T[],
  options: RegistryOptions<T>,
) {
  const byId = new Map<string, T>();
  const bySecondaryKey = new Map<string, T>();

  for (const rawEntry of source) {
    const entry = deepFreeze(rawEntry);
    if (byId.has(entry.id)) throw new Error(`Duplicate ${options.label} id "${entry.id}".`);
    byId.set(entry.id, entry);

    if (options.secondaryKey) {
      const key = options.secondaryKey(entry);
      if (bySecondaryKey.has(key)) throw new Error(`Duplicate ${options.secondaryLabel} key "${key}".`);
      bySecondaryKey.set(key, entry);
    }
  }

  const all = deepFreeze(Array.from(byId.values()));
  return Object.freeze({ all, getById: ..., getBySecondaryKey: ... });
}
```

Why this exists:

1. **Duplicate detection at module load.** If two articles share an id or slug, the *build* fails — not a page, at runtime, for one visitor. Same for two images sharing an id.
2. **O(1) lookups.** `postBySlug("tokyo-...")` is a `Map.get`, not an array scan. Called on every article render and every related-story resolution.
3. **Immutability by construction.** Every entry is deep-frozen before it is stored, and the returned registry object is frozen too. A client component that tries to sort `posts` in place throws instead of silently corrupting the shared array.

Why a factory function and not a class: the registry has no lifecycle, no inheritance, no `this`. A closure over two `Map`s is smaller, cannot be subclassed into something surprising, and returns a frozen object literal that is trivially tree-shaken. See §13.

How the article registry is assembled

```
// app/content/articles.ts
const importedArticles = importedContent.articles as readonly BlogPost[];
const articleCandidates: BlogPost[] =
  Array.from(staticArticles, (a) => attachRegisteredDocuments(a as BlogPost));
articleCandidates.push(...importedArticles.map(attachRegisteredDocuments));

const sortedArticles = articleCandidates.sort(
  (first, second) => publicationTime(second.date) - publicationTime(first.date),
);

const articleRegistry = createContentRegistry(sortedArticles, {
  label: "article",
  secondaryKey: (article) => article.slug,
  secondaryLabel: "article slug",
});

export const posts          = articleRegistry.all;
export const featuredPosts  = deepFreeze(posts.slice(0, 3));
export const archivePosts   = deepFreeze(posts.slice(featuredPosts.length));
export const latestPosts    = deepFreeze(posts.slice(0, 6));
export const categories     = deepFreeze(categoriesForPosts(posts));
export const postById       = articleRegistry.getById;
export const postBySlug     = articleRegistry.getBySecondaryKey;
```

So: **ordering is purely by date, newest first.** `featuredPosts` is the three newest; `archivePosts` is everything after that. There is no editorial pinning today — `featuredRank` exists in the type and the import schema but nothing reads it yet.

The barrel: `app/content.ts`

Components import from `"../content"`, never from `"../content/articles"`. That single re-export file is the public API of the content layer:

```
export { mediaAssets, mediaById, mediaForArticle, mediaLibrary, placeMedia } from
"./content/media-assets";
export { documentAssets, documentById, documentLibrary, documentsForArticle, placeDocument }
from "./content/document-assets";
export { archivePosts, categories, categoryByName, featuredPosts, latestPosts, postById,
postBySlug, posts } from "./content/articles";
export type { BlogPost, BlogSection, MediaAsset, DocumentAsset, ... } from "./content/types";
```

Keeping this boundary means the internal file layout of `app/content/` can change without touching a single component.

Category discovery

`categories.ts` has a hand-ordered preferred list, then appends any *new* category discovered in the articles, alphabetically:

```
export function categoriesForPosts(posts: readonly BlogPost[]) {
  const names = new Set(posts.map((post) => post.category));
  const preferred = preferredCategories.filter((category) => names.delete(category.name));
  const discovered = [...names].sort((a, b) => a.localeCompare(b))
    .map((name) => ({ name, anchor: categoryAnchor(name), navigationLabel: name }));
  return [...preferred, ...discovered];
}
```

Practical effect: importing an article with `"category": "Architecture"` makes an **Architecture** section appear in "Pick a path" with a slugified anchor, with no code change.

7. Where content actually comes from (three sources)

`staticArticles` in `static-articles.ts` is:

```
export const staticArticles = [
  ...localPreviewArticles,      // (3) dev-only, injected by Vite
  { id: "tokyo-skytree-and-shibuya", ... }, // (1) hand-written
  ...
];
```

#	Source	Lives in	Edited by	Present in production?
1	Hand-written articles	<code>app/content/static-articles.ts</code> + <code>media-assets.ts</code> + <code>document-assets.ts</code>	a developer, in TypeScript	yes
2	Imported articles	<code>content/index.json</code> (+ per-article files)	the importer only	yes
3	Local preview fixture	<code>app/content/local-preview-article.ts</code>	a developer, for QA	no

Why source 3 is interesting

Real QA files (a work sample PDF, a reflective report DOCX) contain personal information and must never be deployed. The solution is a **Vite virtual module** whose contents differ between `serve` and `build`:

```
// vite.config.ts
function localPreviewArticles(enabled: boolean): Plugin {
  return {
    name: "local-preview-articles",
    resolveId(id) { if (id === "virtual:local-preview-articles") return "\0virtual:local-preview-articles"; },
    load(id) {
      if (id !== "\0virtual:local-preview-articles") return;
      if (!enabled) return "export const localPreviewArticles = [];;"; // production
      return `import { localPreviewArticle } from
${JSON.stringify(path.resolve("app/content/local-preview-article.ts"))};
      export const localPreviewArticles = [localPreviewArticle];`; // dev
    },
  };
}
// ...
plugins: [ localPreviewArticles(command === "serve"), ... ]
```


The fixture files themselves live in `.local-test-assets/documents/` (gitignored) and are served by a **serve-only** middleware at `/__local-test-documents/` with `no-store`, `nosniff`, byte-range support and a strict basename check. In production the module is literally an empty array — the metadata is not merely inert, it is *absent from the bundle*.

The committed fixture set is four generated demonstration files — a six-slide `.pptx`, a two-page `.docx`, a four-page `.pdf` and a 120-row `.csv` — chosen so every renderer is exercised without any fixture carrying personal or client information. Anything that does carry such information belongs in `.local-test-assets/documents/private/`, which the middleware cannot serve at all: it rejects any name that is not a bare basename, so a subdirectory path never resolves.

8. The article import pipeline

This is the most intricate part of the codebase, and the part most worth understanding before you change anything.

8.1 The promise it makes

Either an article is fully published — canonical record, image files, document files, registry entry, receipt — or nothing at all changed on disk and your source folder is still there, untouched, with a printed reason.

8.2 The flow

```
imports/articles/my-story/
  article.md or article.json
  images/*.jpg
  documents/*.xlsx
    |
    | 1. DISCOVER      run-imports.mjs → discoverArticlePackages()
    ▼
  is it a real directory (not a symlink) containing article.json|article.md?
    |
    | 2. LOCK          .article-import/import.lock (exclusive "wx" open)
    ▼
    | 3. PARSE         markdown-package.mjs (if .md) → same shape as article.json
    ▼
    | 4. VALIDATE      validation.mjs – whitelist every field, reject unknowns
    ▼
    | 5. PLAN          import-package.mjs → validateAndPlan()
    |                  • sha256 every file
    |                  • sharp-probe every image (real format + dimensions)
    |                  • JSZip/PDF/UTF-8 structural check every document
    |                  • allocate deterministic IDs
    |                  • assert no ID/slug/file collision
    ▼
    | 6. STAGE          write everything into .article-import/transactions/<id>/stage/
    ▼
    | 7. COMMIT         rename() staged dirs into place (atomic on one filesystem)
    |                  then writeJsonAtomic(content/index.json)
    ▼
    | 8. RECEIPT        .article-import/receipts/<importId>.json
    ▼
    | 9. CLEANUP        rm -r imports/articles/my-story
    ▼
  content/articles/art_....json + public/images/articles/art_.../ + index.json updated
```

8.3 Discovery — where packages are found

`run-imports.mjs` accepts four discovery modes, in priority order:

Mode	Trigger	Use
packagePaths	internal	the watcher passes exact stable directories
packageNames	<code>--package my-story</code>	import one named package from the inbox
sourcePaths	<code>--source imports/batch-x</code>	import a specific batch folder
default	no flags	<code>imports/articles/*</code> plus every other <code>imports/<batch>/</code> (auto-descending into <code>import-ready/</code> if present)

Directories starting with `_` or `.` are skipped — that is why `imports/articles/_template/` is never imported.

A "package" is defined structurally, and symlinks are rejected at every level:

```
async function isArticlePackage(candidate) {
  const candidateStats = await lstat(candidate).catch(() => null);
  if (!candidateStats?.isDirectory() || candidateStats.isSymbolicLink()) return false;
  const [jsonStats, markdownStats] = await Promise.all([
    lstat(path.join(candidate, "article.json")).catch(() => null),
    lstat(path.join(candidate, "article.md")).catch(() => null),
  ]);
  return Boolean((jsonStats?.isFile() && !jsonStats.isSymbolicLink())
    || (markdownStats?.isFile() && !markdownStats.isSymbolicLink()));
}
```

8.4 Locking

Only one import may run at a time. The lock is an exclusive file create (`"wx"` fails if the file exists), with a one-hour staleness escape so a crashed run does not wedge the project forever:

```
const handle = await open(lockPath, "wx"); // throws if another import holds it
...
if (lockStats && Date.now() - lockStats.mtimeMs > 60 * 60 * 1000) {
  await rm(lockPath, { force: true });
  return acquireLock(root); // adopt the stale lock
}
```

Dry runs skip the lock entirely (`dryRun ? async () => {} : await acquireLock(root)`).

8.5 Validation — `validation.mjs`

This module is a hand-written validator with a strict, **whitelist** philosophy. The crucial helper:

```
function exactKeys(value, allowed, location) {
  for (const key of Object.keys(value)) {
    if (!allowed.includes(key)) throw new Error(`${location} contains the unsupported field "${key}".`);
  }
}
```

An unknown field is an **error**, not something ignored. A typo like "sumary" fails immediately with the exact path, rather than publishing an article with no summary.

What it enforces, in summary:

Rule	Detail
schemaVersion	must be exactly 1
importId , ids, keys	/^[a-z][a-z0-9_-]{2,80}\$/
slug	lowercase kebab-case, ≤100 chars
date	"D Month YYYY" and must parse to a real date
accent	one of sky , coral , teal , navy
art	one of house , gift , shelf , type , weekend , city
sections	1–40; each needs at least one of paragraphs/list/references/images/documents/quote
section id	kebab-case, unique within the article
paragraphs	≤80 per section, ≤12 000 chars each
afterParagraph	integer, −1 or a real paragraph index
references	must be a parseable http : https: URL
assets	≤30 per package; every declared key must be placed, and every placed key must be declared
documents	≤12; same two-way key check
tags	1–20
relatedArticleIds	≤12, each must already exist in the index
text normalisation	every string is .normalize("NFC").trim() ed

That two-way asset check deserves emphasis:

```
for (const reference of references) if (!keys.has(reference))
  throw new Error(`Article references undeclared asset key "${reference}".`);
for (const key of keys) if (!references.includes(key))
  throw new Error(`Asset key "${key}" is declared but never placed in the article.`);
```

You cannot ship an orphan image, and you cannot reference an image you forgot to include.

8.6 Planning and file inspection — `import-package.mjs`

Declared metadata is never trusted. Each file is opened and inspected:

Images — real format and dimensions come from `sharp`, not the filename:

```
const metadata = await sharp(source.path).metadata();
const extension = supportedFormats.get(metadata.format ?? "");
if (!extension || !metadata.width || !metadata.height)
  throw new Error(`Asset ${asset.key} is not a supported JPEG, PNG, WebP or AVIF image.`);
if (metadata.width * metadata.height > MAX_IMAGE_PIXELS)
  throw new Error(`Asset ${asset.key} exceeds the 200 megapixel safety limit.`);
```

Documents — three different structural checks by type:

```
if (extension === ".csv") {
  // must contain no NUL bytes and must decode as strict UTF-8
  new TextDecoder("utf-8", { fatal: true }).decode(buffer);
}
if (extension === ".pdf") {
  // must start with %PDF- and contain %%EOF in the last 4 KB
}
// .pptx/.docx/.xlsx: open as a ZIP, then require
// [Content_Types].xml + ppt/presentation.xml | word/document.xml | xl/workbook.xml
// and reject >10 000 entries, >200 MB expanded (zip-bomb guard),
// and any entry path that is absolute or contains ".."
```

Limits enforced during planning:

Limit	Value
Single image	25 MB
Single document	50 MB (CSV: 10 MB)
Whole package	250 MB
Decoded image pixels	200 megapixels
OpenXML entries / expanded size	10 000 / 200 MB

ID allocation is deterministic, so re-running the same package produces the same IDs:

```
function generatedId(prefix, source) {
  return `${prefix}_${sha256Text(source).slice(0, 24)}`;
}
// article: generatedId("art", importId) → art_036d0a4179dd79db7d9dff25
// asset:   generatedId("asset", `${importId}:${key}`) → asset_76f7fd2cd692707b2f05200a
// document: generatedId("doc", `${importId}:${key}`) → doc_...
```

You may also pin an `id` explicitly in the manifest; the generated one is only a default.

8.7 Idempotency — running the same import twice

Every committed import records a **digest** (of the resolved manifest + file checksums) and a **sourceDigest** (of format + raw manifest bytes + file checksums) under its `importId` in `content/index.json`.

```
if (previousImport) {
  const sourceMatches = previousImport.sourceDigest
    ? previousImport.sourceDigest === sourceDigest
    : previousImport.digest === digest;
  if (!sourceMatches)
    throw new Error(`Import ID "${manifest.importId}" was already used by a different
package.`);
  return { alreadyCommitted: true, ... };
}
```

Behaviour:

- **Same importId, byte-identical files** → the importer verifies the canonical files still exist, then just deletes the redundant inbox copy. Status: `cleaned`.
- **Same importId, different content** → hard error. This is the guard against accidentally overwriting a published article by re-using an import id.

8.8 The transaction — staging, journal, atomic commit

Nothing is written into `content/` or `public/` until everything has been built somewhere private:

```
const transactionRoot = path.join(root, ".article-import/transactions", transactionId);
const stageRoot = path.join(transactionRoot, "stage");
// ... write article JSON, copy images, copy documents, write asset metadata ... all into stage
await updateJournal(journalPath, journal, "staged");

await rename(stagePublicDir, targetPublicDir);      // atomic directory moves
await rename(stageMetadataDir, targetMetadataDir);
await rename(stageArticle, targetArticle);
await updateJournal(journalPath, journal, "files-moved");

await writeJsonAtomic(indexPath, nextIndex);        // temp file + rename
indexCommitted = true;
await updateJournal(journalPath, journal, "index-committed");
```

The journal records the state machine: `preparing` → `staged` → `files-moved` → `index-committed` → `complete` (or `rolled-back`, or `cleanup-pending`). If a failure happens **before** the index commit, every target is removed and the journal is marked `rolled-back`. After the index commit, the only remaining step is deleting the inbox folder; if that fails you get status `imported-with-warning` and the article is still correctly published.

Two more safety properties:

- `copyExclusive` uses `copyFile(src, dst, 1)` — `COPYFILE_EXCL`, which **fails** rather than overwrites.
- `writeJson` uses `{ flag: "wx" }` — same principle for JSON.
- Before staging, every destination path is checked for existence and for containment inside the repo root (`isWithin`).

8.9 Path safety — `filesystem.mjs`

```
export function assertInside(parent, candidate, label) {
  const relative = path.relative(parent, candidate);
  if (!relative || relative === ".") return;
  if (relative.startsWith("..") || path.isAbsolute(relative))
    throw new Error(`${label} escapes its allowed directory.`);
}

export async function safePackageFile(packageDir, relativePath, label) {
  if (typeof relativePath !== "string" || !relativePath.length ||
  path.isAbsolute(relativePath))
    throw new Error(`${label} must be a relative file path.`);
  const resolved = path.resolve(packageDir, relativePath);
  assertInside(packageDir, resolved, label);
  const stats = await lstat(resolved).catch(() => null);
  if (!stats?.isFile() || stats.isSymbolicLink())
    throw new Error(`${label} must reference a regular, non-symbolic file.`);
  return { path: resolved, stats };
}
```

Absolute paths, `../` traversal, and symlinks are all rejected — a package cannot reach outside itself to read `/etc/passwd` or to plant a file outside `public/`.

8.10 The Markdown adapter — `markdown-package.mjs`

`article.md` is the friendly authoring format. The adapter converts it to exactly the same manifest shape `article.json` produces, then hands it to the same validator — so Markdown gets zero special treatment downstream.

Supported syntax:

Markdown	Becomes
YAML frontmatter (<code>key: value</code> only — no nesting, no lists-of-maps)	article fields
<code>## Heading</code>	a new section; <code>id</code> is the slugified heading, de-duplicated with <code>-2</code> , <code>-3</code>
blank-line-separated text	a paragraph (soft-wrapped lines are joined with a space)
<code>![alt](images/foo.jpg)</code>	an asset, auto-keyed from the filename, placed after the current paragraph
a line of <code>*italic text*</code> right after an image	that image's caption
<code>[Label](documents/foo.xlsx)</code> on its own line	a document attachment (only for the five allowed extensions, and only relative paths)
<code>1.</code> / <code>-</code> lists	<code>list + listStyle</code>
a <code>-</code> list where every item is <code>[Label](https://...)</code>	<code>references</code> instead of a list
<code>> quote</code>	the section's <code>quote</code>

Defaults it fills in: `importId` → `markdown-<slug>-v1`; `accent` → `sky`; `art` → a per-category default (`Travel` → `city`, `Culture` → `shelf`, ...); `artLabel` → the uppercased category; `tags` → the category if none given; `date` accepts ISO `2026-07-23` and converts to `23 July 2026`.

`MARKDOWN_ADAPTER_VERSION = 3` is recorded in the receipt, so you can tell which adapter generation produced a given article.

8.11 The watcher and `npm run dev`

`npm run dev` runs `dev-with-imports.mjs`, which supervises two child processes: `npm run dev:site` (the vinext dev server) and `watch-article-imports.mjs`. The watcher restarts itself if it dies; killing the site kills everything.

The watcher polls every 1200 ms and uses a **two-stable-scans** rule so a package being copied in is never imported half-written:

```
const fingerprint = await packageFingerprint(packageDir); // sha256 of name:size:mtime for every file
const previous = stability.get(packageDir);
const stableScans = previous?.fingerprint === fingerprint ? previous.stableScans + 1 : 1;
stability.set(packageDir, { fingerprint, stableScans });
if (stableScans >= 2 && rejected.get(packageDir) !== fingerprint) ready.push(packageDir);
```

A package that fails is recorded in `rejected` **keyed by its fingerprint** — so it is not retried in a hot loop; it is retried only once one of its files actually changes. After a successful import the watcher re-runs image optimisation, and Vite hot-reloads `content/index.json`.

Watcher state is written to `.article-import/watcher.json` (`starting` / `watching` / `importing` / `error` / `stopped`) — useful when debugging "why has my article not appeared".

9. The image pipeline

9.1 Generation — `scripts/optimize-images.mjs`

Runs automatically via `predev`, `prebuild`, and after each successful import. It walks `public/images/**`, and for each source produces AVIF and WebP at `[480, 768, 1200, 1600]` plus the original width — skipping any width larger than the source:

```
const widths = [...responsiveWidths.filter((width) => width < metadata.width), metadata.width];
for (const width of widths) {
  for (const format of ["avif", "webp"]) {
    const output = path.join(outputRoot, `${basename}-${width}.${format}`);
    if (await writeVariant(source, output, width, format)) generated += 1;
  }
}
```

`writeVariant` is incremental — it compares mtimes and skips work that is already current, so a rebuild with no new images costs nothing. Output lands in `public/_optimized/images/`, which is **gitignored**: variants are derived data, regenerated on every machine.

`.rotate()` is applied first so EXIF-rotated phone photos come out upright.

9.2 Consumption — `media/responsive-image.tsx`

```
function optimizedPath(src: string, width: number, format: "avif" | "webp") {
  const extensionIndex = src.lastIndexOf(".");
  const basename = extensionIndex === -1 ? src : src.slice(0, extensionIndex);
  return `/_optimized${basename}-${width}.${format}`;
}

export function ResponsiveImage({ image, className, loading = "lazy", sizes: sizesOverride,
fullResolution = false }) {
  const sizes = fullResolution ? `${image.width}px` : sizesOverride ?? (image.portrait
    ? "(max-width: 640px) calc(100vw - 40px), 540px"
    : "(max-width: 940px) calc(100vw - 40px), 900px");
  return (
    <picture className={className}>
      <source type="image/avif" srcSet={sourceSet(image, "avif")} sizes={sizes} />
      <source type="image/webp" srcSet={sourceSet(image, "webp")} sizes={sizes} />
      <img src={image.src} width={image.width} height={image.height} alt={image.alt}
        loading={loading} decoding="async"
        fetchPriority={loading === "eager" ? "high" : undefined} />
    </picture>
  );
}
```

The naming convention is the entire contract: `/images/tokyo/skytree-view.jpg` implies `/_optimized/images/tokyo/skytree-view-1200.avif`. If you ever move images, both the `src` in the asset record and the generated tree move together, because the generator mirrors the directory structure.

The original `<img src>` remains the un-optimised file — a guaranteed fallback for any browser that supports neither AVIF nor WebP, and the target of "View full resolution".

10. The document (attachment) pipeline

Attachments — a spreadsheet of the chronology, a deck, a reading brief — render as a card inside the article, and open a full preview dialog in the browser. No upload, no server-side conversion; the file is parsed in the reader's own browser by Web Workers.

The feature is split by responsibility so that **adding an attachment never requires editing renderer UI**:

File	Responsibility
<code>document-contract.ts</code>	Validates and freezes document records. The MIME/extension/path/size invariants.
<code>document-assets.ts</code>	The registry: hand-written + imported documents, grouped by article.
<code>document-placements.ts</code>	Where each hand-written document appears (article + section + paragraph).
<code>document-format.ts</code>	Label + icon policy (<code>pptx</code> → "PowerPoint" / presentation).
<code>article-document.tsx</code>	The in-article card with Preview and Download .
<code>document-preview-dialog.tsx</code>	The modal: focus trap, Escape, theme sync, portal.
<code>document-preview-header.tsx</code>	Title, caption, Download / Open file / Close.
<code>document-preview-workspace.tsx</code>	Lazy viewer + Suspense + error boundary + status overlay.
<code>document-preview-status.tsx</code>	The loading and error panels.
<code>document-preview-state.ts</code>	Pure reducer for <code>loading</code> / <code>ready</code> / <code>error</code> + retry attempt.
<code>document-viewer.tsx</code>	The actual <code><FileViewer></code> (default export, lazily imported).
<code>document-viewer-options.ts</code>	Immutable Flyfish configuration and vendor asset URLs.
<code>document-viewer-error-boundary.tsx</code>	Class component; contains renderer crashes.
<code>document-viewer-loader.ts</code>	The single shared <code>import()</code> , reused for preloading.

10.1 The contract

```
export function assertDocumentAsset(asset: DocumentAsset): void {
  const suffix = `.${asset.extension}`;
  if (!asset.filename.toLocaleLowerCase("en-US").endsWith(suffix))
    throw new Error(`Document "${asset.id}" filename does not match its ${asset.extension}
format.`);
  if (asset.mimeType !== DOCUMENT_MIME_TYPES[asset.extension])
    throw new Error(`Document "${asset.id}" has an invalid MIME type for ${asset.extension}.`);
  if (!asset.src.startsWith("/") || asset.src.includes(".."))
    throw new Error(`Document "${asset.id}" must use a safe root-relative source URL.`);
  if (!Number.isSafeInteger(asset.size) || asset.size <= 0)
    throw new Error(`Document "${asset.id}" must declare a positive byte size.`);
}
```

Every document — hand-written or imported — passes through this at module load. A wrong MIME type is a build failure, not a broken preview for a reader.

10.2 Loading strategy (three tiers)

This is the reason the article page stays light despite shipping a document renderer:

```
// Tier 1 – nothing loads with the article. The card is server-rendered.

// Tier 2 – on *intent*, warm the viewer shell:
<motion.aside
  onPointerEnter={preloadDocumentViewer}
  onFocusCapture={preloadDocumentViewer}
  onTouchStart={preloadDocumentViewer}
>

// Tier 3 – on Preview, mount it:
const DocumentViewer = lazy(loadDocumentViewer);
```

`document-viewer-loader.ts` is two lines and exists so tiers 2 and 3 share **one** module promise (a second `import()` of the same module resolves instantly):

```
export const loadDocumentViewer = () => import("./document-viewer");
export const preloadDocumentViewer = () => void loadDocumentViewer();
```

10.3 Viewer configuration

```
export function documentViewerOptions(theme: DocumentViewerTheme): Readonly<FileViewerOptions>
{
  return Object.freeze({
    preset: officePreset,
    rendererMode: "replace",
    theme,
    locale: "en-US",
    styleIsolation: "shadow",          // Shadow DOM – document CSS cannot leak into the site
    toolbar: TOOLBAR_OPTIONS,
    ui: UI_OPTIONS,
    pdf: PDF_OPTIONS,                  // /vendor/pdf/pdf.worker.mjs, cmaps, wasm, fonts
    docx: DOCX_OPTIONS,                // /vendor/docx/docx.worker.js + jszip
    presentation: PRESENTATION_OPTIONS,
    spreadsheet: SPREADSHEET_OPTIONS,
  });
}
```

Two decisions worth calling out:

- `styleIsolation: "shadow"` — a Word document's own styles are rendered inside a shadow root, so an attachment can never restyle the article around it.
- **Explicit absolute** `/vendor/...` **URLs** — not relative. An article lives at `/blog/some-slug`, so relative worker URLs would resolve to `/blog/vendor/...`. The Vite plugin `fileViewerRenderers({ copyAssets: true, chunkStrategy: "renderer" })` populates `public/vendor/` for both dev and build.

The viewer also re-verifies the declared extension against the filename at render time and throws on mismatch — that throw is caught by the error boundary and shown as a retry state, not a white screen:

```
const detectedExtension = normalizeFileExtension(getExtension(document.filename));
if (detectedExtension !== document.extension)
  throw new Error(`Document extension mismatch for ${document.filename}.`);
```

10.4 State machine

```
export function reduceDocumentPreview(state, action) {
  if (action.type === "retry") return Object.freeze({ status: "loading", attempt: state.attempt + 1 });
  if (state.status === action.type) return state;          // no-op re-renders
  return Object.freeze({ ...state, status: action.type });
}
```

`attempt` is not cosmetic — it is part of the viewer's React `key`:

```
const viewerKey = `${asset.id}:${state.attempt}`;  
<DocumentViewer key={viewerKey} ... />
```

Bumping `attempt` unmounts and remounts the viewer, which is the only reliable way to recover from a failed parse. The same key resets the error boundary.

10.5 Theme synchronisation

The dialog subscribes to the `<html data-theme>` attribute with a `MutationObserver`, through `useSyncExternalStore` so server and client agree:

```
function subscribeToTheme(callback: () => void) {  
  const observer = new MutationObserver(callback);  
  observer.observe(document.documentElement, { attributes: true, attributeFilter: ["data-  
theme"] });  
  return () => observer.disconnect();  
}  
  
const theme = useSyncExternalStore<"light" | "dark">(subscribeToTheme, readTheme, () =>  
"light");
```

Toggling the site theme with a preview open re-themes the document immediately.

10.6 Serving headers

`public/_headers` forces the correct `Content-Type`, `Content-Disposition: inline` and `X-Content-Type-Options: nosniff` for every attachment extension in production; `publishedDocumentHeaders()` in `vite.config.ts` replicates exactly that in the dev server so behaviour matches.

11. Routing and pages

Route	File	Notes
/	app/page.tsx	Intro → featured → archive → categories → newsletter
/blog/[slug]	app/blog/[slug]/page.tsx	The article
/photo/[id]	app/photo/[id]/page.tsx	One photograph, full page

All three pre-enumerate their parameters, so every page is known at build time:

```
export function generateStaticParams() { return posts.map(({ slug }) => ({ slug })); }  
// and  
export function generateStaticParams() { return mediaLibrary.map(({ id }) => ({ id })); }
```

The article route

```
export default async function PostPage({ params }: { params: Promise<{ slug: string }> }) {  
  const post = postBySlug(await params.slug);  
  if (!post) notFound();  
  
  const related = post.related  
    .map((reference) => postById(reference) ?? postBySlug(reference))  
    .filter((item) => item !== undefined);  
  
  return (<>  
    <BlogHeader />  
    <main id="main-content">  
      <ArticleView post={post} />  
      <MotionReveal><RelatedStories posts={related} /></MotionReveal>  
      <MotionReveal><NewsletterSignup variant="article" /></MotionReveal>  
    </main>  
    <JournalFooter />  
  </>);  
}
```

Note `postById(reference) ?? postBySlug(reference)` — `related` entries may be **either** an article id or a slug, and unresolvable references are silently dropped rather than crashing the page.

Metadata

`app/layout.tsx` computes `metadataBase` from the incoming request headers so Open Graph URLs are correct on any host (localhost, preview, production) with no env var:


```
const host = incoming.get("x-forwarded-host") ?? incoming.get("host") ?? "localhost:3000";
const protocol = incoming.get("x-forwarded-proto") ?? (host.startsWith("localhost") ? "http" :
"https");
return { metadataBase: new URL(`${protocol}://${host}`), title: { default: "Snoopy HQ Journal",
template: "%s | Snoopy HQ Journal" }, ... };
```

Per-page metadata is derived from the content: the article's title, summary and tags.

The theme boot script

The one piece of inline script in the app, in `<head>` so it runs before first paint and there is no flash of the wr